Symbol Tables

Storing keys and values: simple implementations· Algorithms §3.1

This video

- A detour: why GTA Online took **six minutes** to load
- What a **symbol table** is, and its API
- Two elementary implementations
    - sequential search in a linked list
    - binary search in an ordered array
- **Ordered** operations: `min`, `floor`, `rank`, and friends

A detour: GTA Online load times

GTA Online was famous for **slow loading**: around six minutes on a good PC.

In 2021, a player named t0st took the game apart to find out why. He wrote it up in a blog post: "How I cut GTA Online loading times by 70%".

6 min

loading screen, before

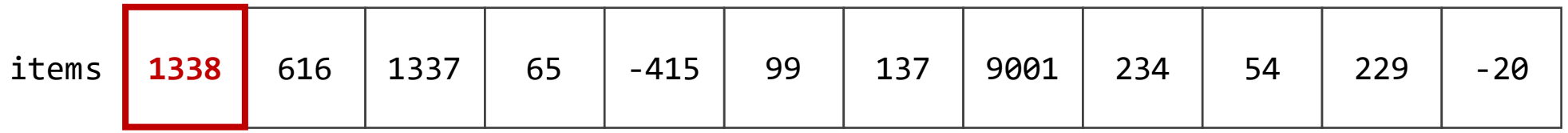
1 min 50 s

after t0st's two fixes

What the loading screen was doing

The game reads a 10 MB file listing everything you can buy: **63,000 items**.

Before storing each item, it checks the **whole list so far** to make sure the item isn't already there.



668 new item: are you 668?

cost so far

compares **1**

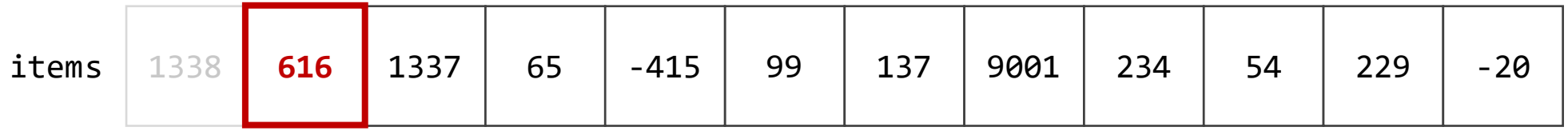
668 vs. 1338

Is 1338 the same item as 668? **No.** On to the next one.

What the loading screen was doing

The game reads a 10 MB file listing everything you can buy: **63,000 items**.

Before storing each item, it checks the **whole list so far** to make sure the item isn't already there.



668 new item: are you 668?

cost so far

compares **2**

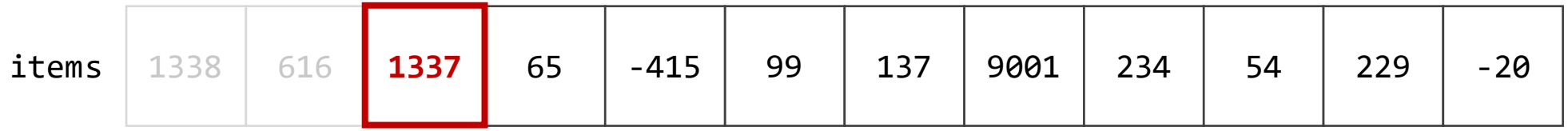
668 vs. 616

616? **No.**

What the loading screen was doing

The game reads a 10 MB file listing everything you can buy: **63,000 items**.

Before storing each item, it checks the **whole list so far** to make sure the item isn't already there.



668 new item: are you 668?

cost so far

compares **3**

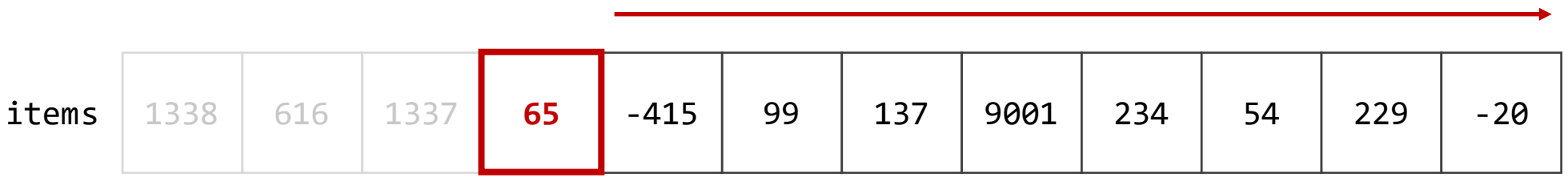
668 vs. 1337

1337? **No.**

What the loading screen was doing

The game reads a 10 MB file listing everything you can buy: **63,000 items**.

Before storing each item, it checks the **whole list so far** to make sure the item isn't already there.



668 new item: are you 668?

cost so far

compares **4**

668 vs. 65 (12 by the end)

65? **No.** And so on, all the way to the end.

The list isn't sorted, so the only way to know 668 isn't there is to check **every item**:
about n checks when the list holds n items.

Adding it up: accidental n^2

Item 1 checks 0 items. Item 2 checks 1. Item 3 checks 2. … Item n checks n − 1.

Total: about **$n^2/2$** checks.

With 63,000 items, t0st counted about **2 billion** checks.

Why didn't anyone notice? With a **small test file**, n^2 and n log n both finish instantly. The difference only shows up at full size.

n	**n log n**	**n^2 / 2**
100	~700	5,000
1,000	~10,000	500,000
63,000	~1 million	**~2 billion**

What should the game have done?

The question it kept asking: **"have I seen this key before?"** That's a **symbol table** question.

- With a hash table (end of this unit), each check takes about **1** step instead of n. All 63,000 items: about 63,000 steps, not 2 billion.

Rockstar paid t0st a 10,000 dollar bug bounty for finding and fixing this bug. Pretty much everyone thinks he deserved more.

What is a symbol table?

A collection of **key–value pairs**. **Put** a value in with a key; later, **get** the value back by giving the key.

application	key	value
dictionary	word	definition
book index	term	page numbers
DNS	domain name	IP address
compiler	variable name	type and value
GTA Online	item's hash	the item

Also called a **map**, **dictionary**, or **associative array**.

Python's `dict` is one. If you've used a `dict`, you've used a symbol table.

(An array is a symbol table whose keys are 0, 1, …, n − 1.)

The API

```
1  public class ST<Key, Value>
2
3      ST()                                // create an empty symbol table
4      void put(Key key, Value val)        // put a key-value pair in the table
5      Value get(Key key)                  // value paired with key (or null)
6      void delete(Key key)                // remove key and its value
7      boolean contains(Key key)           // is there a value paired with key?
8      boolean isEmpty()                   // is the table empty?
9      int size()                          // number of key-value pairs
10     Iterable<Key> keys()                // all the keys in the table
```

put and **get** are the two that matter most: they're the ones we'll measure for every implementation.

Key and Value are placeholder types, like Key in MaxPQ. Here there are two of them.

Conventions

- **Values are never `null`.**
- `get()` returns **`null`** if the key isn't there.
- `put()` on a key that's already there **replaces** the old value.

Two nice consequences:

- `contains()` is one line (right).
- `put(key, null)` can mean "delete this key". That's called **lazy** deletion.

Python's `dict.get()` does the same thing: a missing key gives back None.

```
1  public boolean contains(Key key)
2  {  return get(key) != null;  }
3
4  public void delete(Key key)
5  {  put(key, null);  }
```

A test client: 13 calls to `put`

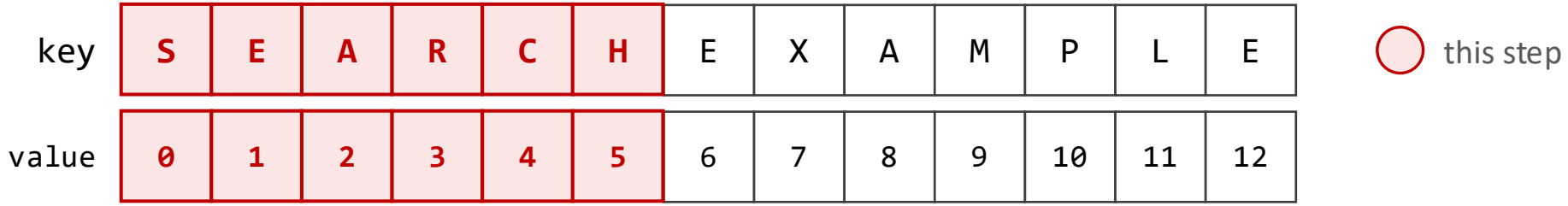
key	S	E	A	R	C	H	E	X	A	M	P	L	E
value	0	1	2	3	4	5	6	7	8	9	10	11	12

```
1  String[] keys = "S E A R C H E X A M P L E".split(" ");
2  ST<String, Integer> st = new ST<String, Integer>();
3  for (int i = 0; i < keys.length; i++)
4      st.put(keys[i], i);
```

Each letter is put with its position in the string. (Same `split` trick as `SortClient` in HW4.)

Let's watch the table after each `put`.

Test client: the first six puts



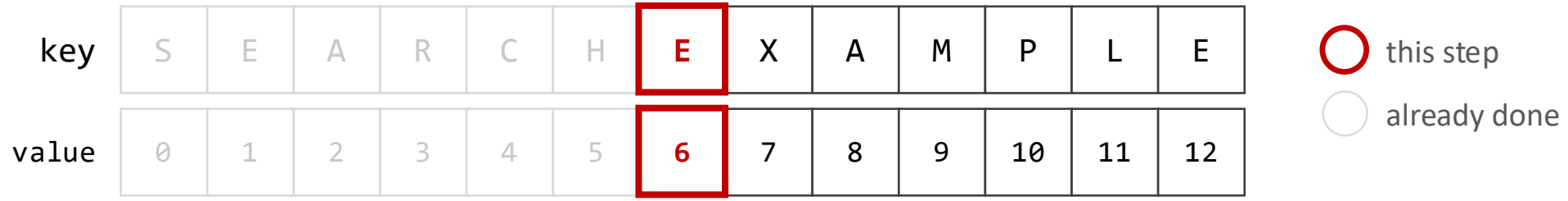
key	value
S	0
E	1
A	2
R	3
C	4
H	5

S, E, A, R, C, H: none of them is in the table yet, so each put adds a **new key**.

6 puts, 6 keys.

(The table is shown in the order the keys arrived. A symbol table doesn't promise any particular order.)

Test client: `put(E, 6)`



key	value
S	0
E	**1 → 6**
A	2
R	3
C	4
H	5

E is **already in the table**.

`put` doesn't add a second E. It **replaces** E's value: 1 becomes 6.

Still 6 keys.

Test client: the next five `puts`



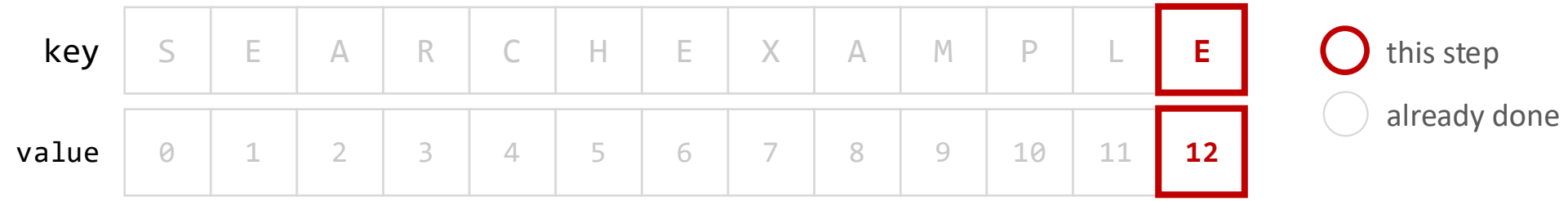
key	value
S	0
E	6
A	**2 → 8**
R	3
C	4
H	5
X	**7**
M	**9**
P	**10**
L	**11**

X, M, P, L are new: 4 more keys.

A is already there, so its value is replaced: 2 becomes 8.

10 keys.

Test client: `put(E, 12)`



key	value
S	0
E	**6 → 12**
A	8
R	3
C	4
H	5
X	7
M	9
P	10
L	11

E one more time: 6 becomes **12**.

13 puts, 10 keys. Each key keeps the value from its **last** `put`.

These are the keys and values we'll trace for the rest of this video, and in the next two.

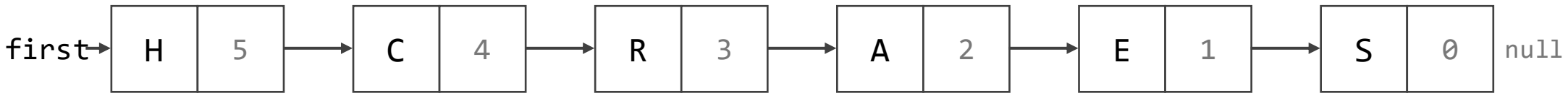
A real client: counting words

```
for (String word : words) {                    // each word
    if (!st.contains(word)) st.put(word, 1);   // new: 1
    else st.put(word, st.get(word) + 1);       // seen: +1
}
```

Project Gutenberg text	words	different
A Tale of Two Cities	~140,000	10,000+
Shakespeare, complete works	~900,000	tens of thousands

Every word is a `get` and a `put`. If those are slow, this program is slow.

Symbol table attempt 1: sequential search

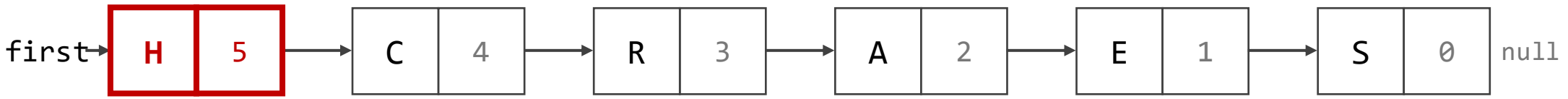


Keep the pairs in a linked list, **in no particular order**. Each node holds a key, a value, and a link.

- **`get`**: walk the list, comparing with `equals()`, until the key matches. Reach `null`? Not there.
- **`put`**: walk the list. Key there? Replace its value. Not there? Add a **new node at the front**.

This is the list after putting S E A R C H with values 0–5: each new key went on the front, so H is first.

get(R)



Start at `first`. Is it H? **No.** Follow the link to the next node.

cost so far

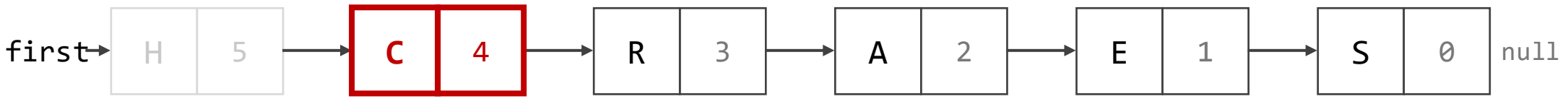
compares **1**

R vs. H

the node we're checking

already checked: not a match

get(R)



Is it C? **No.** Follow the link.

cost so far

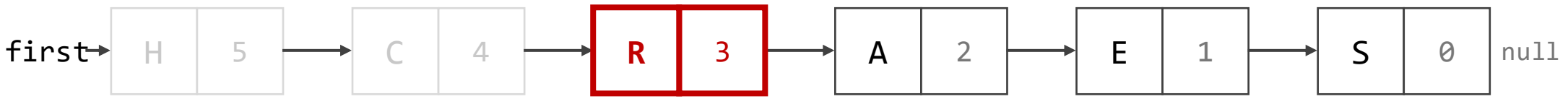
compares **2**

R vs. C

the node we're checking

already checked: not a match

get(R)



Is it R? **Yes!** Return its value, 3.

A search hit stops as soon as it finds the key. On average that's about halfway: n/2 compares.

A million keys: about **500,000** compares, on average.

cost so far

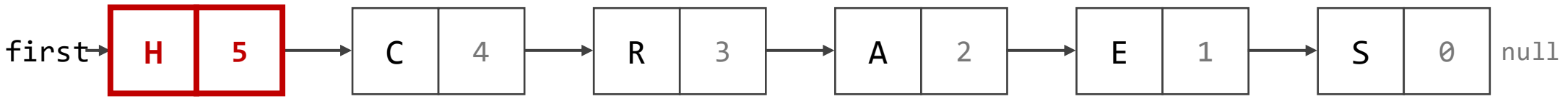
compares **3**

R vs. R: found it

the node we're checking

already checked: not a match

put(X, 7)



Is X already here? Start at `first`. H? **No.** Follow the link.

cost so far

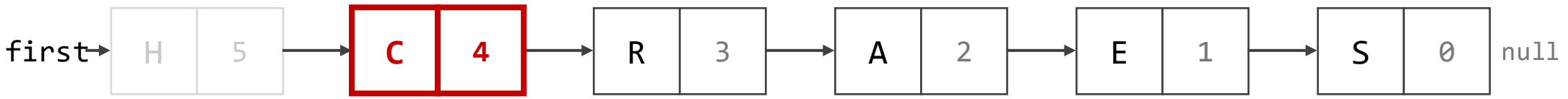
compares **1**

X vs. H

the node we're checking

already checked: not a match

put(X, 7)



C? **No.** Follow the link.

cost so far

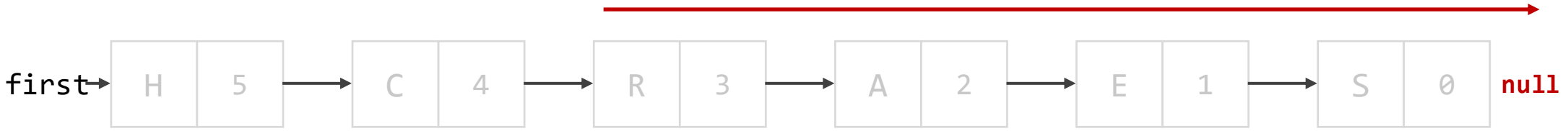
compares **2**

X vs. C

the node we're checking

already checked: not a match

put(X, 7)



R? A? E? S? **No**, **no**, **no**, **no**. Then we reach `null`: X isn't here.

A search miss always checks every node: n compares. There's no way to stop early, because the list isn't in order.

A million keys: a **million** compares, for every new key.

cost so far

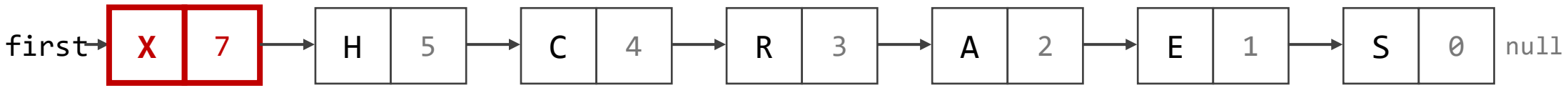
compares **6**

+4: R, A, E, S

the node we're checking

already checked: not a match

put(X, 7)



Not there, so X goes on the **front**: one new node, two links changed.

Adding the node is fast. **Finding out** we needed to add it took n compares.

cost so far

compares 6

adding X: no compares

the new node

Sequential search: the code

```
1  public Value get(Key key) {
2      for (Node x = first; x != null; x = x.next)
3          if (key.equals(x.key))
4              return x.val;             // search hit
5      return null;                      // search miss
6  }
7
8  public void put(Key key, Value val) {
9      for (Node x = first; x != null; x = x.next)
10         if (key.equals(x.key)) {
11             x.val = val;              // already there: replace
12             return;
13         }
14     first = new Node(key, val, first);   // new: add to front
15 }
```

Node has three fields: `key`, `val`, `next`.

The only thing we ask of a key is `equals()`. Any type works.

`put` is a search, plus a little.

Sequential search: costs

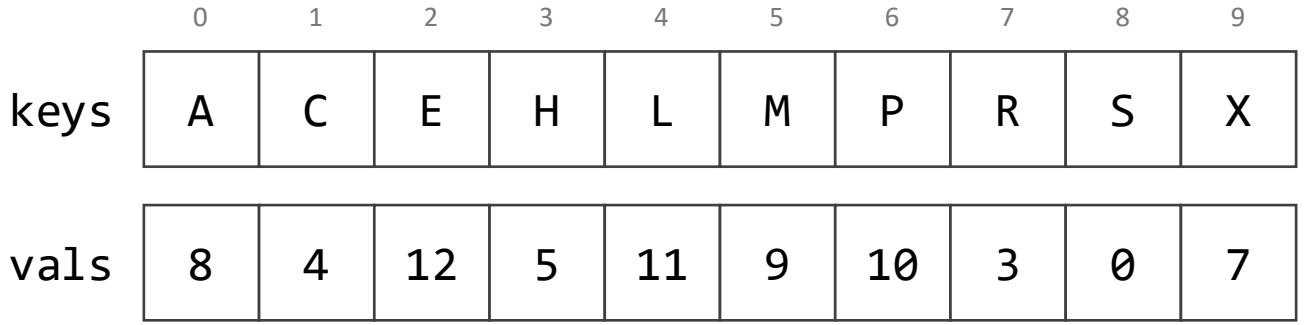
implementation	search	insert	avg. search hit	avg. insert
sequential search	n	n	n / 2	n

Every `put` of a new key searches the whole list first.

So putting n different keys into an empty table costs 0 + 1 + 2 + ... + (n − 1) ≈ $\mathbf{n^2/2}$ compares.

That's the bug from the GTA Online loading screen.

Attempt 2: binary search



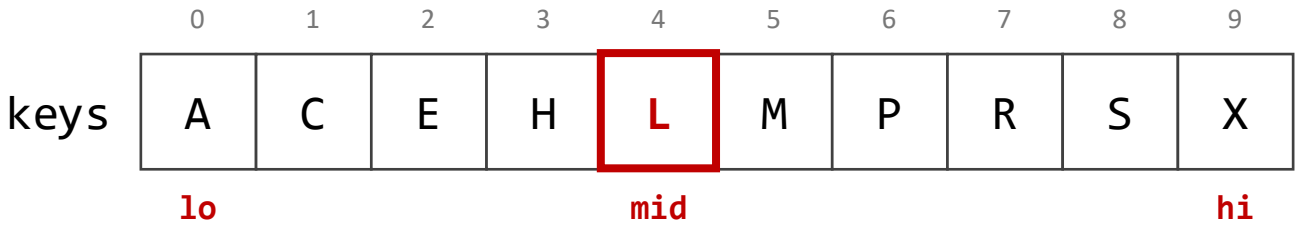
These are the 10 keys from the test client, sorted, with their final values.

Two **parallel arrays**, keys kept in **sorted order**. `vals[i]` belongs to `keys[i]`.

Keys must be `Comparable`: we need `compareTo()` to keep them in order.

Everything is built on one helper, `rank(key)`: **how many keys are less than key?** It uses binary search.

`rank(P)`: a search hit



cost so far

compares **1**

P vs. L

`lo = 0`, `hi = 9`, so `mid = 4`.

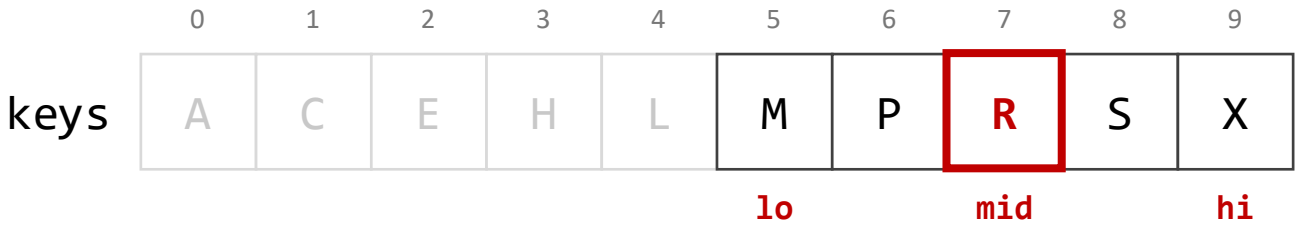
Compare P with `keys[4]`, which is L.

P is **greater**. P can't be anywhere in 0..4, so `lo = 5`.

`keys[mid]`, the key we compare with

ruled out

`rank(P)`: a search hit



`lo = 5`, `hi = 9`, so `mid = 7`: R.

P is **less** than R, so `hi = 6`.

Half of what was left is gone again.

cost so far

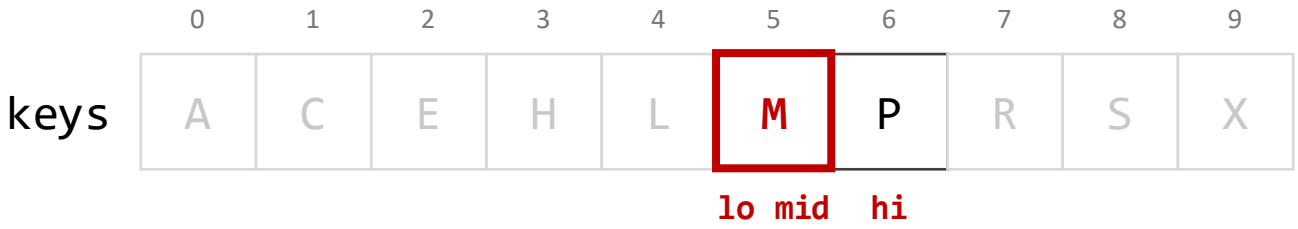
compares **2**

P vs. R

`keys[mid]`, the key we compare with

ruled out

`rank(P)`: a search hit



`lo = 5`, `hi = 6`, so `mid = 5`: M.

P is **greater** than M, so `lo = 6`.

cost so far

compares **3**

P vs. M

keys[mid], the key we compare with

ruled out

`rank(P)`: a search hit



cost so far

compares **4**

P vs. P: equal

`lo = hi = mid = 6`: P.

Equal! Return **6**.

Check: 6 keys are less than P (A C E H L M). That's P's rank.

4 compares for 10 keys. A million keys: about **20**.

keys[mid], the key we compare with

ruled out

`rank(Q)`: a search miss



cost so far

compares **4**

same path as P: L, R, M, P

Q follows the **same path** as P: L, R, M, P.

At P, Q is greater, so `lo = 7`. Now `lo` is past `hi`, so the loop ends. Return **`lo`**, which is **7**.

Not found, but 7 is still useful: **7 keys are less than Q**, so 7 is exactly **where Q would go**.

ruled out

Binary search: `rank` and `get`

```
1  public int rank(Key key) {
2      int lo = 0, hi = n - 1;
3      while (lo <= hi) {
4          int mid = lo + (hi - lo) / 2;
5          int cmp = key.compareTo(keys[mid]);
6          if      (cmp < 0) hi = mid - 1;
7          else if (cmp > 0) lo = mid + 1;
8          else return mid;
9      }
10     return lo;
11 }
```

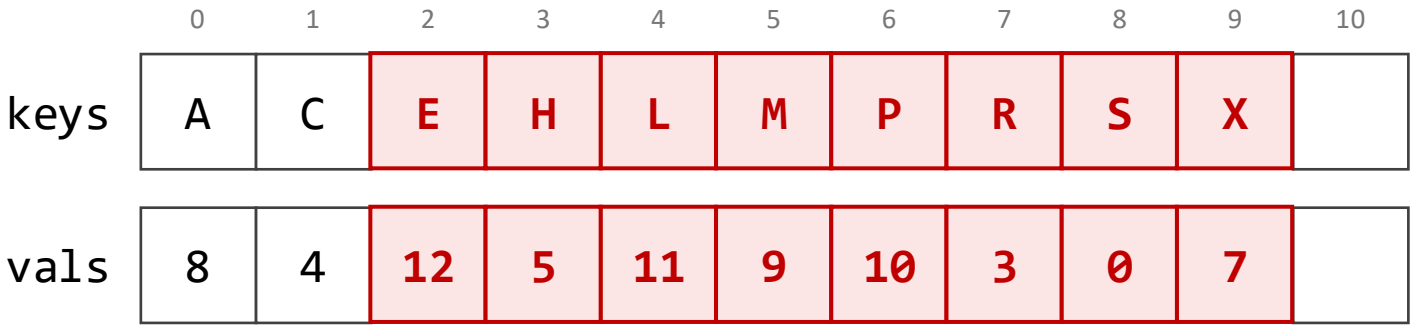
```
1  public Value get(Key key) {
2      int i = rank(key);
3      if (i < n && keys[i].compareTo(key) == 0) return vals[i];
4      return null;
5  }
```

Line 10: a miss returns `lo`, the **number of smaller keys**.

`get` asks `rank` where the key would be, then checks whether it's actually there.

`i < n` matters: a key bigger than everything has rank n, which is off the end.

put(D, 13)



cost so far

compares	4
moves	0

3 in rank(D), 1 to check slot 2

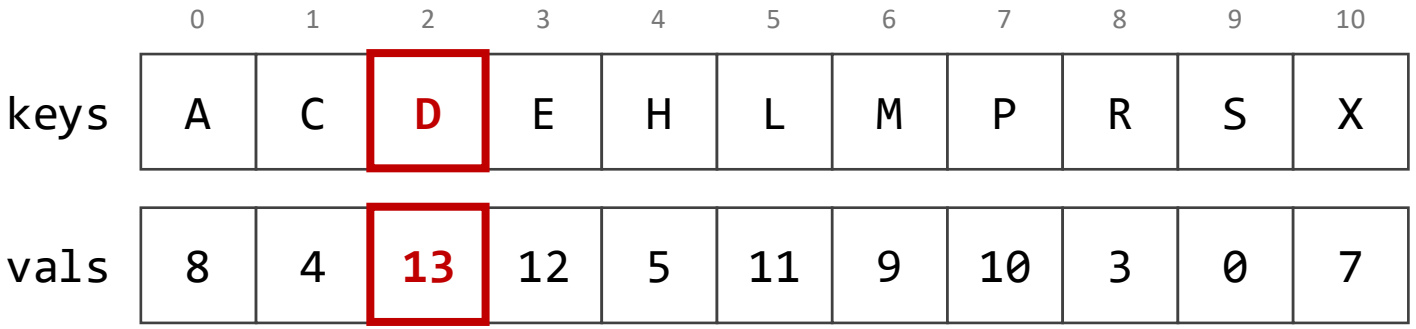
`rank(D)` is **2**: only A and C are smaller.

D isn't there, so it has to go in slot 2.

Everything from slot 2 on has to **move right one** to make room: 8 keys, and their values.

has to move

put(D, 13)



cost so far

compares	4
moves	**8**

+8 keys slide right (vals too)

D is in, and the keys are still sorted.

Finding the spot: **lg n** compares.
Making room: up to **n** moves.

On average, a new key lands in the middle, so about **n/2** keys move.

A million keys: about **500,000** moves for every new key.

Binary search: put

```
1  public void put(Key key, Value val) {
2      int i = rank(key);
3      if (i < n && keys[i].compareTo(key) == 0) {
4          vals[i] = val;                  // already there
5          return;
6      }
7      for (int j = n; j > i; j--) {       // make room
8          keys[j] = keys[j-1];
9          vals[j] = vals[j-1];
10     }
11     keys[i] = key;
12     vals[i] = val;
13     n++;
14 }
```

Lines 7–10 are the slow part: they slide every larger key one slot right.

Look familiar? It's **one pass of insertion sort**.

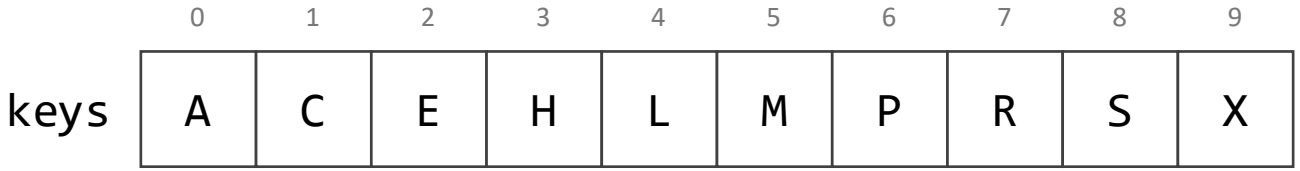
Full array: double it, like the resizing-array stack.

Ordered symbol tables

Keeping the keys sorted made `put` slow. It does give us an advantage, though: if the keys are `Comparable`, we can ask questions about **order**

```
Key min()                    // smallest key
Key max()                    // largest key
Key floor(Key key)           // largest key <= key
Key ceiling(Key key)         // smallest key >= key
int rank(Key key)            // number of keys < key
Key select(int k)            // key of rank k
void deleteMin()             // delete smallest key
void deleteMax()             // delete largest key
int size(Key lo, Key hi)     // number of keys in [lo..hi]
Iterable<Key> keys(Key lo, Key hi)   // keys in [lo..hi], in sorted order
```

Ordered operations on our keys



call	returns
`min()`	A
`max()`	X
`floor("G")`	E
`ceiling("G")`	H
`floor("H")`	H

call	returns
`rank("P")`	6
`rank("Q")`	7
`select(3)`	H
`size("D", "N")`	4
`keys("D", "N")`	E H L M

G isn't a key. Its floor is the key just below where it would be; its ceiling, just above.

H **is** a key, so it's its own floor: "or equal".

D and N aren't keys either. Ranges work anyway.

In an ordered array, `rank` does the work

```
1  public Key floor(Key key) {
2      int i = rank(key);
3      if (i < n && keys[i].compareTo(key) == 0)
4          return keys[i];                 // key is its own floor
5      if (i == 0) return null;            // nothing is <= key
6      return keys[i-1];
7  }
8
9  public Key select(int k)  {  return keys[k];  }
10 public Key min()           {  return keys[0];  }
```

operation	sequential search	binary search
`min`, `max`, `select`	n	1
`floor`, `ceiling`, `rank`	n	lg n
keys in order	n lg n	n

Sorted order makes these almost free: `select(k)` is just `keys[k]`.

`floor` and `ceiling` are a `rank` plus a check.

The unordered list has to look at everything, every time.

A design note: only offer the operations people need. Every method in an API is a promise you have to keep.

Each attempt so far is slow at some things

implementation	search	insert	avg. search hit	avg. insert	ordered ops	keys need
sequential search	n	n	n / 2	n	n	`equals()`
binary search	lg n	n	lg n	n / 2	lg n	`compareTo()`
what we want	**lg n**	**lg n**	**lg n**	**lg n**	**lg n**	

Sequential search is slow at everything. Binary search is fast at **search** and at the **ordered operations**. But **insert** still has to shift up to n keys.

This is fine for large-ish data sets. But with **millions** of different keys, every new key shifts about a million others.

(Hash tables, at the end of this unit, will beat all of these at search and insert, on average. But they scramble the keys, so no ordered operations)

Coming up: binary search trees

Can we get **lg n** for search, insert, *and* the ordered operations? Yes: **binary search trees**.

Binary search is fast because each compare **throws away half** the keys.

The ordered array is slow because the keys are packed into **one block of memory**: inserting means shifting.

Next video: a **linked** structure where every compare still throws away about half, but inserting just means **adding a node**.